Research Project –1(ID4093)

IPDIMS - CONFERENCE WEBSITE MANAGEMENT SYSTEM

Research Project-1 submitted to the
National Institute of Technology Rourkela
In partial fulfilment of

Bachelors in technology

in

Industrial Design

By

Vishal Kumar Yadav (122ID0893)
Jatoth Rajesh (122ID0636)

Under the supervision of
Prof. B.B.V.L. Deepak

Department of Industrial Design

National Institute of Technology Rourkela

Table of contents

1. Introduction 1
2. Background 2
3. Motivation 3
4. Literature Review 4
5. Objective 5
6. Methodology 6
7. Detailed Design 12
8. Work done 15
9. Work to be done 17
10. Conclusion 17

References 18

Acknowledgement

I take this opportunity to express my heartfelt gratitude to Prof. B.B.V.L. Deepak, Department of Industrial Design, NIT Rourkela, for his guidance and constant support throughout the development of my project, IPDIMS - Conference Website Management System. His suggestions at various stages helped me structure the problem, explore possible solutions, and complete the project with clarity and confidence.

I am also thankful to the Department of Industrial Design for providing the required resources, laboratory facilities, and a productive learning environment, which played a key role in the successful completion of this work.

Abstract

In most conferences, institutional research programs, and academic events, it is observed that managing academic paper submissions and coordinating review workflows is generally time consuming and requires a lot of manual effort. To address this challenge, the project presents the design and development of the IPDIMS - Conference Website Management System, which is a web-based system that streamlines the complete lifecycle of paper submission, assignment, review, and feedback dissemination.

The system enables three independent user roles: **Author, Reviewer, and Admin**, each with clearly defined functionality. Authors can sign up, upload their submissions, track the review status, and receive feedback. Admins manage the reviewers, assign submissions, monitor progress, and control the acceptance decisions. Reviewers get the assigned paper(s), evaluate them using a structured interface, and submit feedback and recommendations. Technologies include React for User/Admin dashboards, Node.js + Express as backend, MongoDB for database storage, JWT authentication, and Cloudinary for secure document handling. Automatic e-mail notifications of status updates ensure that everything is transparent and in real time.

By automating repetitive tasks such as assigning reviewers, tracking the status of work, and sending notifications, IPDIMS reduces administrative overhead and prevents delays in the review cycle. The system provides a structured, secure, and efficient workflow; hence, it is appropriate for academic events, research groups, and institutional documentation processes. Future enhancements may include automated reviewer suggestion using interest-matching algorithms and analytics dashboards for decision support.

1. Introduction

Research conferences and academic events usually rely on the submission and peer reviewing process of technical papers. However, most institutions still manage this workflow manually by using emails, shared drives, and spreadsheets to track submissions, assign reviewers, collect feedback, and communicate decisions. Because this is generally a manual process, files get misplaced, delayed responses, missing clarity, and uncalled-for dependence on the organizing authority. committee.

Addressing the challenges cited above, this study proposes IPDIMS - Conference Website Management System is a web-based platform that digitizes and automates the entire submission–review–feedback cycle. The system provides separate dashboards for Authors, Reviewers, and Administrators, therefore ensuring role-specific access and a streamlined workflow. Authors can upload research papers, track their review status, and get feedback or decisions without having to send multiple emails. Reviewers can access the papers assigned to them, submit evaluations, and recommend acceptance or revision.

Administrators have full oversight, managing reviewers, assigning submissions, updating decisions and also notification. The platform has been developed using the modern MERN stack architecture. React + Vite for user and admin interfaces, Node.js + Express for server side logic, and MongoDB for database operations. Features such as JWT-based authentication, Cloudinary file storage, while notifications over email enhance security, document handling, and allow for efficient communication.

By centralizing document management and automating repetitive tasks, IPDIMS reduces administrative effort, eliminates communication gaps, and accelerates the decision-making. process. The system aims to provide an organized, transparent, and efficient experience for academic events, internal research evaluations, and institutional publication workflows.

2. Background of the work

Academic conferences, research symposia, and institutional review processes all rely on the submission, vetting, and approval of research papers. Traditionally, such workflows have been laboriously managed by email exchange, local storage of files, and manual tracking of documents using spreadsheets. While workable, these methods inevitably lead to bottlenecks: papers disappear into extensive email threads, reviewers experience delays when receiving the documents, authors find the feedback unclear or coming too late, and administrators have no overview of the current status of reviews.

With increased digitization and remote collaboration, there is an emerging need for a structured, centralized, automated platform to support the entire submission-to-review lifecycle. EasyChair and OpenConf offer partial solutions but are either complicated for a first-time user or lack transparency and customization for institution-level research processes.

To address these gaps, this project IPDIMS: Conference Website Management System proposes a simplified digital workflow based on roles, where:

- **Authors** can submit papers and track decisions,
- **Reviewers** can access assigned submissions and provide feedback,
- **Admins** can manage users, assign reviewers, update status, and send notifications.

The system uses the MERN stack, implementing React with Vite, Node.js, Express.js, and MongoDB; it also has secure JWT-based authentication, Cloudinary-based file management, and email-triggered notifications for seamless communication. IPDIMS thus provides transparency with reduced administrative effort through automation in assigning reviewers, tracking the status, and disseminating feedback.

The system is positioned at the crossroads of human-computer interaction and workflow automation to change the face of academic submission management from manual and fragmented to efficient, structured, and user-oriented.

3. Motivation

The motivation of developing IPDIMS stems from the increasing need for efficiency, transparency, and automation of academic paper management processes. At most conferences, research events, and institutional review systems, submissions are still made via email, shared drives, and manually maintained tracking spreadsheets. This fragmented approach results in frequent delays, difficulty in managing multiple submissions, lack of accountability during reviewer assignments, and communication gaps between authors, reviewers, and administrators. As the volume of research submissions continues to grow, these manual workflows become even more inefficient and error-prone.

While available, the existing tools are mostly overly complex for small-scale academic events or often lack the customization required by universities and departments. This points out the clear gap and highlights the need for systems that are streamlined, intuitive, and adaptable to institutional requirements.

IPDIMS aims to solve this problem by offering a single platform that automates the complete submission and review lifecycle. Through role-based dashboards, authors can track submission status, reviewers can evaluate assigned papers, and administrators can manage the entire process from a single interface. Technologies used include JWT for secure access, Cloudinary for file handling, and real-time email notifications, all of which enhance the user experience and improve the reliability of the system.

In a nutshell, the goal is to minimize manual workload, improve coordination, and enable a structured, transparent, and professional academic workflow process. By bridging the gap between administrative efficiency and digital automation, IPDIMS contributes to a more efficient and modern research ecosystem.

4. Literature Review

1. Belchior, Schwabe, and Parreiras (2012) propose an approach that embeds Role-Based Access Control (RBAC) directly into the web application design phase, instead of implementing it after development. Their work extends the Semantic Hypermedia Design Method (SHDM), treating roles, permissions, and access policies as primary components of the application model. By using RDF and N3Logic rules, the system automatically generates Access Control Lists (ACLs), enabling validation and enforcement of authorization during runtime. This reduces security administration complexity and ensures that access policies remain consistent as the system evolves. [1]

2. According to the Auth0 Developer Documentation (2024), role-based authentication in modern web systems relies on stateless token-based authentication mechanisms rather than traditional session-based methods. JSON Web Tokens (JWT) securely store user identity and role information, allowing servers to validate users without maintaining session data. JWT-based authentication improves scalability, particularly in distributed or microservice architectures, and is widely adopted due to its lightweight nature and minimal overhead. [2]

3. Cloudinary Documentation (2024) emphasizes that cloud-based storage is more efficient than database-level file storage for large media or document uploads. Cloudinary provides secure upload APIs, URL-based retrieval, automatic file optimization, and version control. Research indicates that cloud file storage reduces database load, enhances system performance, and enables scalable document handling making it suitable for applications involving frequent uploads or large file sizes. [3]

4. MongoDB Documentation (MongoDB Inc., 2024) highlights that MongoDB's flexible JSON document model is ideal for systems requiring dynamic and hierarchical data structures, such as submission–review workflows. MongoDB enables storage of nested data without complex joins, improving data retrieval performance. Prior studies show that MongoDB supports high scalability, fast indexing, and efficient query performance, making it suitable for document-centric and workflow-driven applications. [4]

5. Objective of the project

The objective of this project is to design and develop IPDIMS, an integrated platform that digitizes and streamlines the complete workflow of academic paper submission and review. The system aims to replace the existing manual and email-based processes with an automated, transparent, and role-based solution that enhances efficiency for all stakeholders. Through this platform, authors can submit papers and track status updates, reviewers can access assigned submissions and provide structured feedback, and administrators can manage users, assign reviewers, update decisions, and monitor progress from a centralized dashboard. Secure document handling through cloud storage, automated email notifications, and role-based authentication form the backbone of the system, ensuring smooth communication and data security. Ultimately, the project focuses on reducing administrative effort, improving coordination, and enabling a seamless digital environment for academic paper management.

6. Methodology

The IPDIMS development follows a user–workflow-driven design philosophy where the process of academic paper handling is mapped to a seamless digital experience. The system integrates frontend–backend communication, database management, and secure role-based authentication, which would guarantee that paper submissions, assigned reviewers, and returned feedback dissemination occurs in an effective and transparent manner. The methodology supported is agile: development principles - enabling refinement continuously by iterative testing and feedback.

6.1 conceptualization

The conceptual foundation of IPDIMS emerged from analyzing how research paper submissions are currently handled in academic settings through email threads, shared documents, and manual reviewer assignment. This approach often results in misplaced files, communication gaps, and delays in feedback.

The key conceptual objective was to create a **centralized digital platform** where:

- Authors can upload papers and track their status,
- Reviewers can evaluate assigned submissions and provide feedback,
- Administrators can manage users, assign reviewers, and update decisions.

The concept emphasizes automation, usability, and transparency, making academic submission management more straightforward and effective.

The conceptual flow is automated as follows:

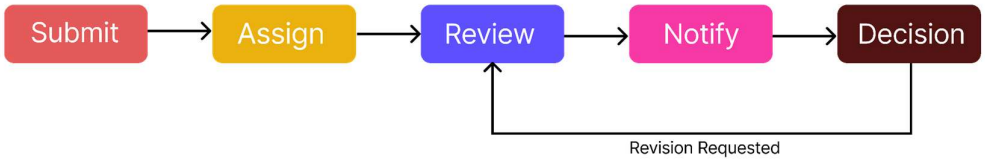

Fig.1 Conceptual flow

6.2 Points to Focus

6.2.1 Role-Based Workflow and Authentication

The system implements secure authentication using JSON Web Tokens (JWT) to ensure that only authorized users can access specific features based on their role Author, Reviewer, or Admin. Middleware is used to validate tokens for each API request, preventing unauthorized access. This ensures that authors can only manage their submissions, reviewers can only access their assigned papers, and administrators have complete control over system operations.

6.2.2 Submission and Document Management

The platform enables authors to upload research papers through an intuitive interface. Documents are processed using Multer and stored securely on Cloudinary instead of being saved directly in the database, ensuring fast access and efficient storage. Each submission is associated with metadata such as submission date, title, keywords, and author details, making search and filtering more efficient for administrators.

6.2.3 Reviewer Assignment and Feedback Mechanism

The admin assigns submissions to reviewers through an integrated dashboard. Reviewers can download the assigned document, evaluate it, and provide structured feedback directly in the system. The feedback includes comments and recommendations such as “Accepted,” “Rejected,” or “Revision Required.” Once feedback is submitted, the author receives real-time status updates.

6.2.4 Email Notification and Automation

Automated email notifications are triggered at key stages of the workflow using Nodemailer/SMTP. Notifications are sent when an author submits a paper, when a reviewer is assigned, and when feedback or a decision is published. This automation eliminates manual follow-ups and ensures that users remain informed throughout the review cycle.

6.2.5 Testing, Validation, and Refinement

Testing, Validation, and Refinement Functionality testing and integration testing were performed throughout development to ensure seamless interaction between front-end interfaces, back-end APIs, and the database. It included testing like submission upload, reviewer assignments, authentication checks, and email automation. Problems found during the tests such as failed uploads, incorrect routing, and role access conflicts were iteratively resolved to enhance reliability and user experience.

6.3 System Design & Architecture

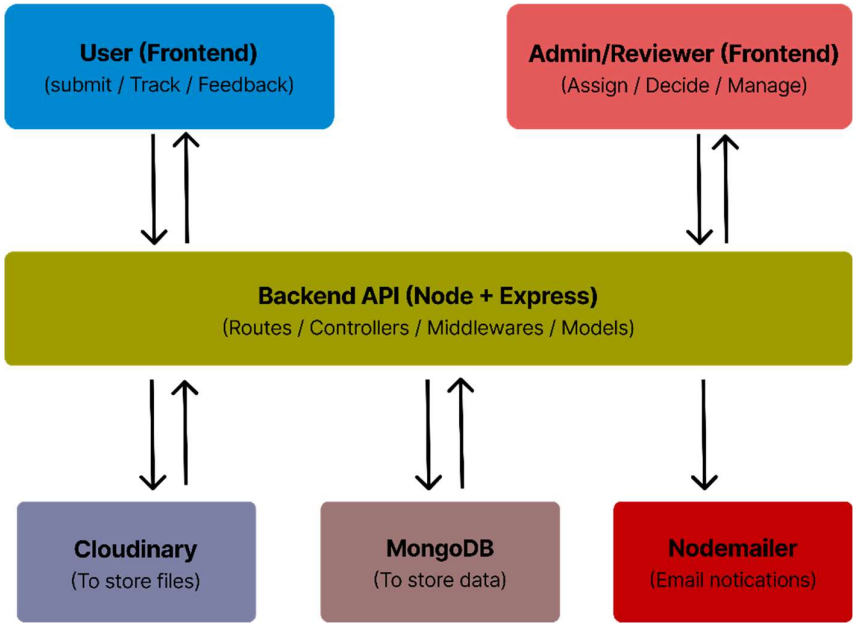

Fig. 2 Architecture

6.4 Wireframe / UI Flow

Fig. 3 Homepage

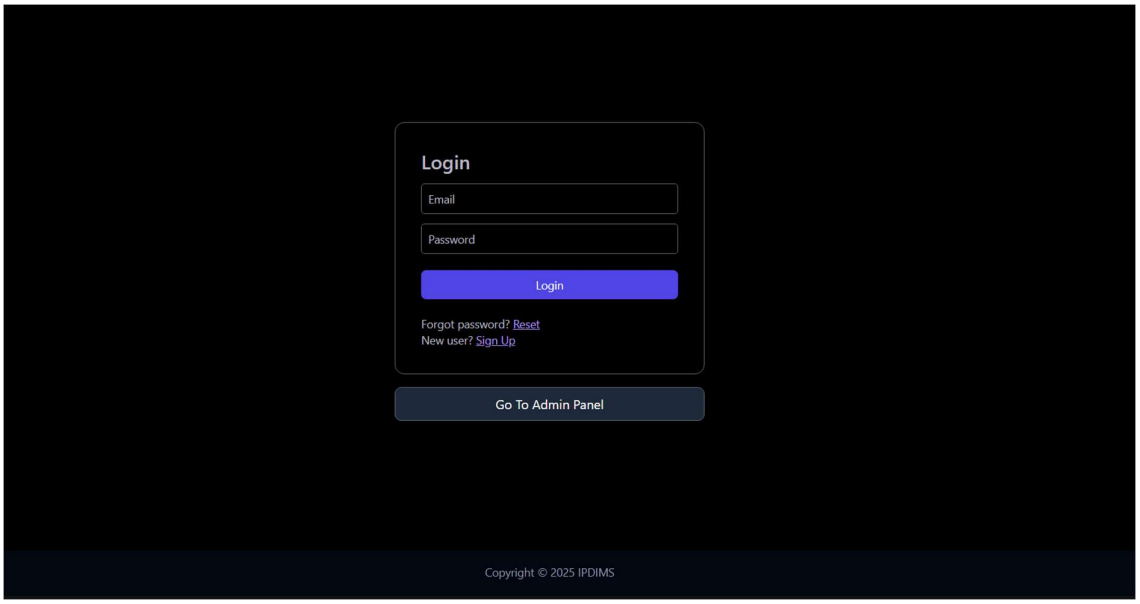

Fig.4 Login page (Author)

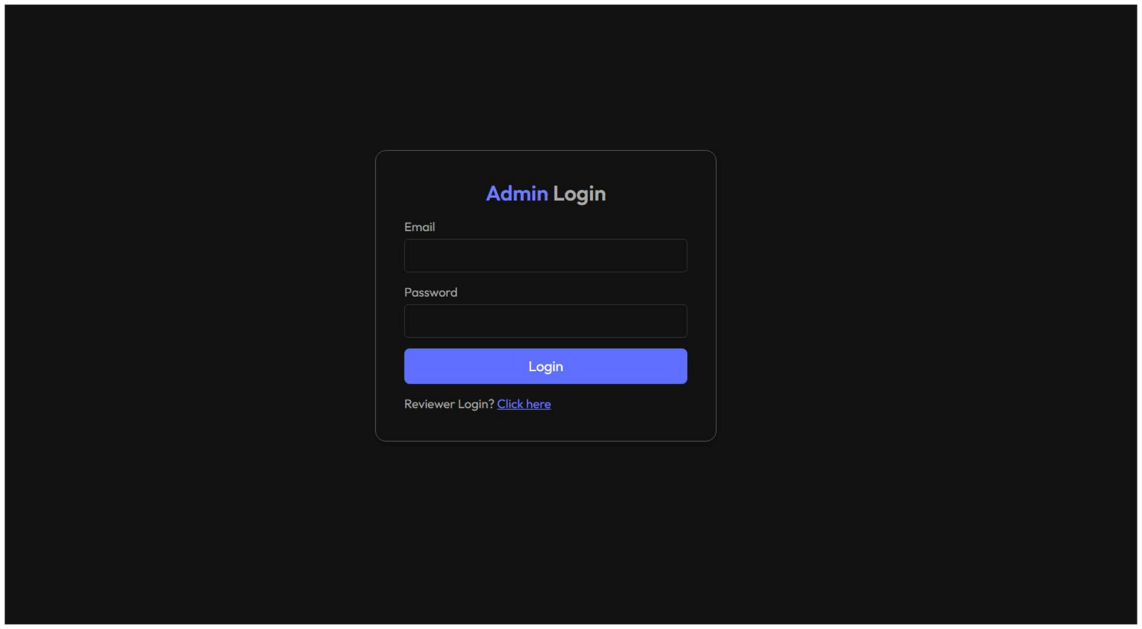

Fig.5 Login page (Admin/ Reviewer)

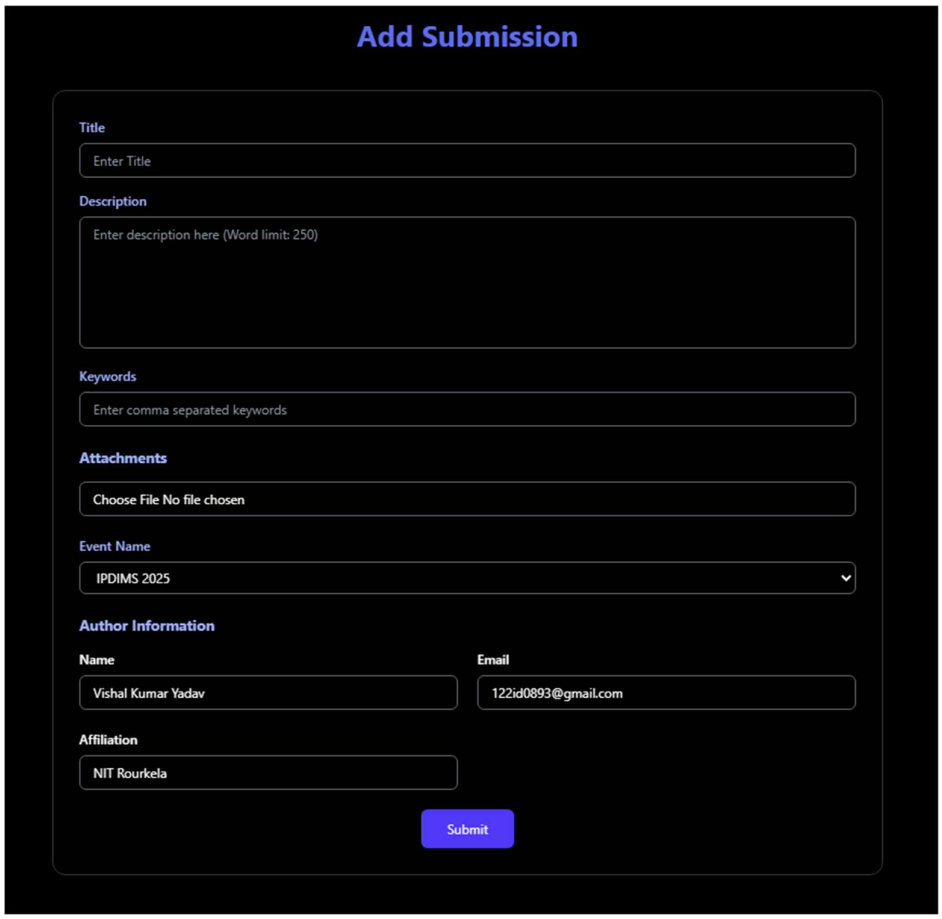

Fig.6 Submission form (Author)

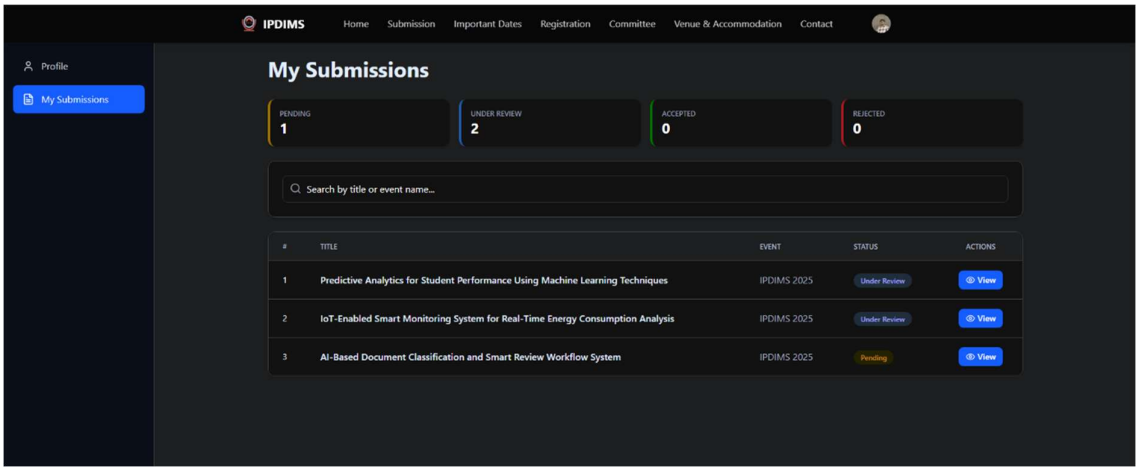

Fig.7 All submission (Author)

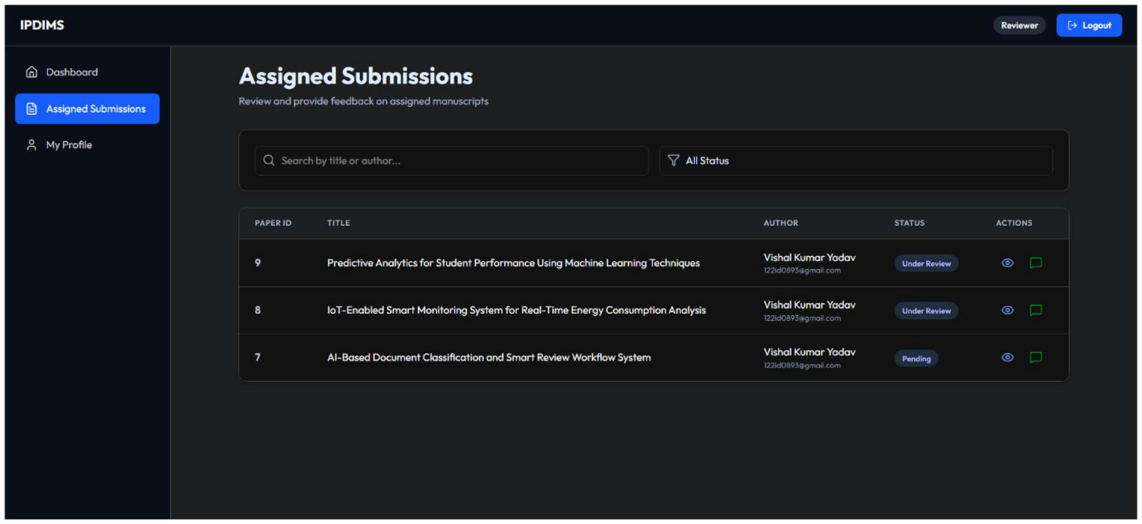

Fig.8 Assigned submission (Reviewer)

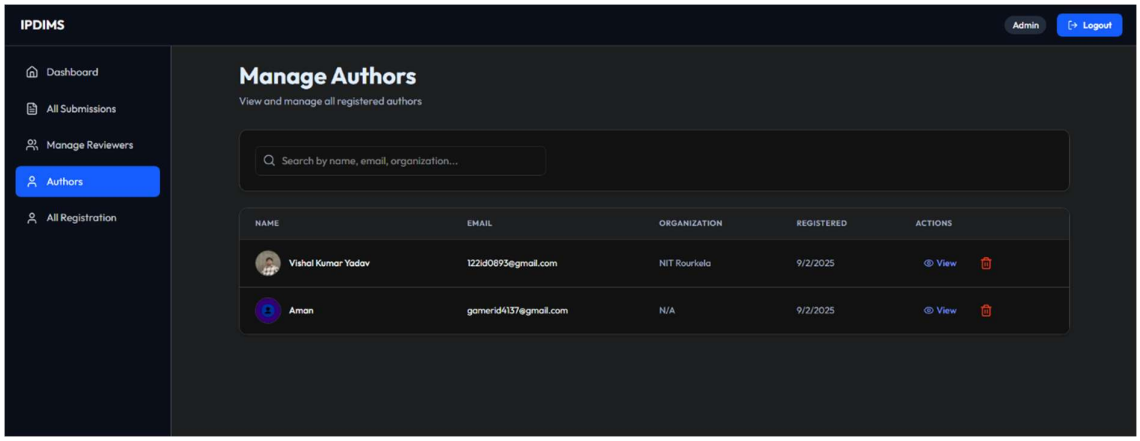

Fig.9 Manage authors (Admin)

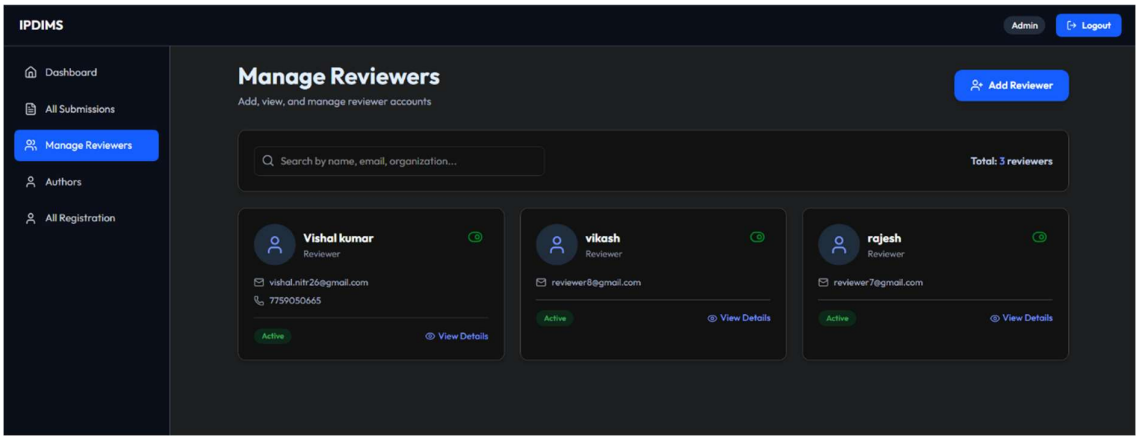

Fig.10 Manage Reviewers (Admin)

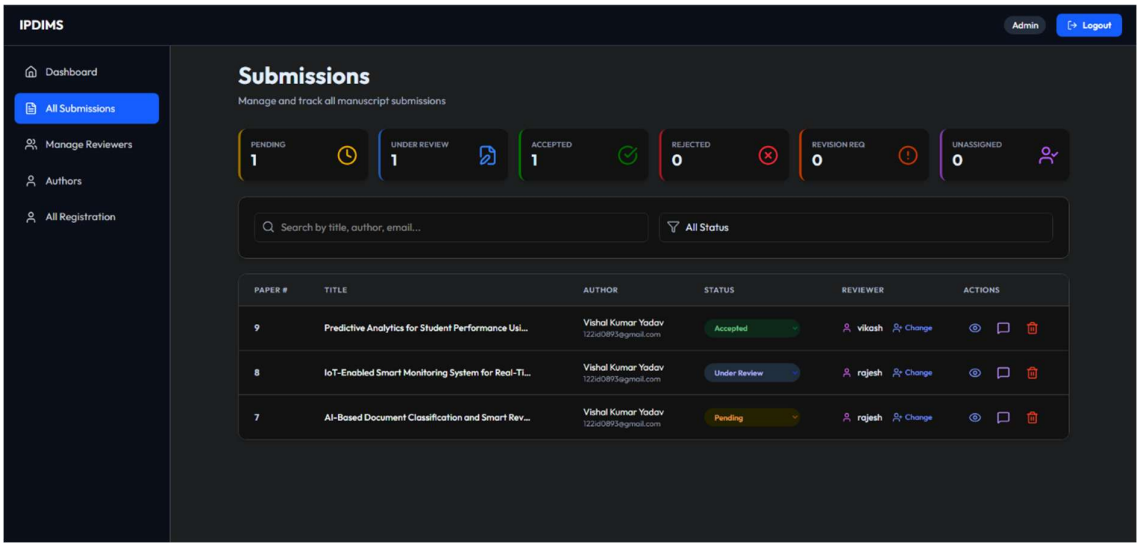

Fig.11 All Submission (Admin)

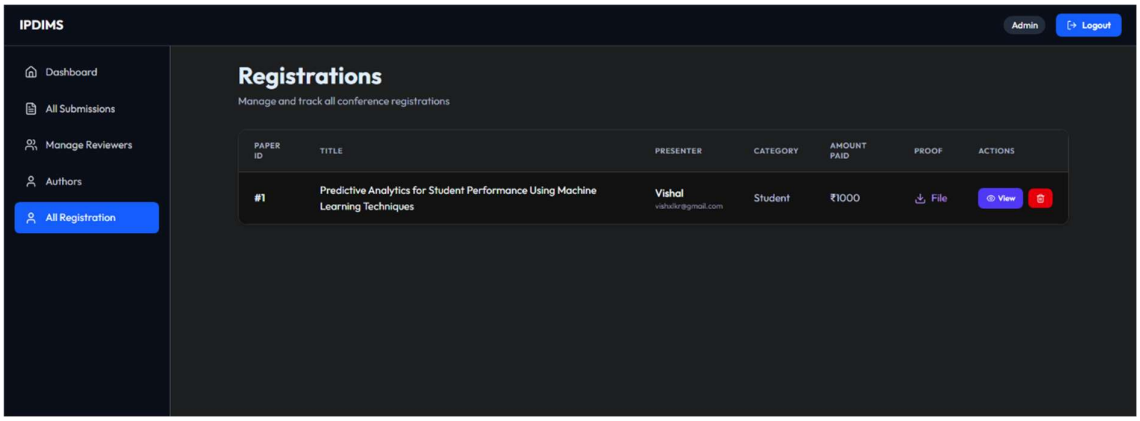

Fig.12 All Registration (Admin)

7. Detailed Design

7.1 Tech Stack used

Component	Technology Used	Description
Frontend	React.js + Vite	Building user dashboard and admin interface
Backend	Node.js + Express.js	Handling API requests and server-side logic
Database	MongoDB	Storing users, submissions, reviewers & records
Authentication	JWT (JSON Web Token)	Secured login with role-based access control
Email Service	Nodemailer (SMTP)	Sending automated emails (submission status etc.)
File Uploading	Multer + Cloudinary	Uploading and storing research paper files
Version Control	Git + GitHub	Code versioning and project tracking
Styling	CSS / Custom Styling	Designing UI using CSS and manual styling

Fig 13 Tech stack

7.2 Backend Design (Server & API Layer)

The backend of IPDIMS is developed using **Node.js and Express.js**, following the MVC (Model-View-Controller) pattern. This ensures clean separation between business logic, data access, and routing.

Key Design Decisions

- All business logic is grouped in *controller files* (e.g., adminController.js, reviewerController.js, submissionController.js).
- All API endpoints are structured into *route files* (e.g., /admin, /reviewer, /user routes).
- Middleware is used to handle authentication, file uploads, and access control.

```
server/
│
├── controllers/      # Handles business logic
├── routes/           # Maps API endpoints to controllers
├── models/           # MongoDB schema structures
├── middlewares/      # JWT auth, multer upload, role authorization
├── config/           # DB connection, Cloudinary, email service
└── server.js         # Entry point of backend
```

Fig.14 Backend Folder structure

7.3 Database Design (MongoDB Schema Layer)

MongoDB stores all persistent data including user profiles, submissions, reviews, and registrations. The database is structured into multiple collections, each representing a logical entity.

7.3.1 User Schema

The User Schema stores personal and login details of authors. The data stored includes name, email, phone, password, gender, designation, personal URL, organization or affiliation, address, bio, and profile image.

7.2.2 Submission Schema

The Submission Schema stores information related to a submitted research paper. It includes auto-generated paper ID, title, description or abstract, keywords, author reference, author name, author email, author affiliation, and uploaded paper file link. It also includes reviewer reference, submission status (Pending, Under Review, Accepted, Rejected, Revision Required), and stored reviewer feedback containing comment, decision, and timestamp.

7.2.3 Reviewer Schema

The Reviewer Schema contains details of individuals who evaluate papers. Stored data includes reviewer name, email, phone, password, designation, organization, specialization areas, bio, address, and gender. It also maintains a list of assigned submissions, active or inactive status, and last paper assignment timestamp.

7.3.4 Registration Schema

The Registration Schema stores participant registration and payment details after paper acceptance. It contains paper ID, paper title, presenter type, participant name, designation, email, mobile number, registration category, registration type, amount paid, payment date, transaction reference number, and payment proof. Additional fields include accommodation requirement, food preference, additional notes, and linked user reference.

7.4 Frontend Design (Client & Admin Dashboards)

The frontend is developed as **two separate React + Vite applications** to isolate responsibilities and maintain a clean role-based architecture.

7.4.1 Client Interface (Author Portal)

This interface is used **only by Authors**. Authors can log in, upload submissions (PDF), track submission status (Pending, Under Review, Accepted, Rejected, Revision Required), and view feedback shared by reviewers. The interface is intentionally minimal, focusing only on paper submission and status tracking.

7.4.2 Admin Interface (Reviewer + Admin Dashboard)

This interface is shared by **Reviewers and Admins**. Reviewers can view assigned submissions, download files, and submit feedback. Admins can view all submissions, assign reviewers, change submission status (Accept, Reject, Revision Required), and manage users, registrations, and reviewers.

7.5 Authentication (JWT + Role-Based Access)

Authentication is handled using **JWT (JSON Web Token)**. When a user logs in, the backend verifies the credentials and generates a token that contains the user's ID and role (Author, Reviewer, or Admin). This token is stored on the frontend and attached to every request sent to the backend.

Each protected route is checked using middleware. The token is first validated, and then role-based access control ensures that users can only perform actions allowed for their role Authors can upload and view their own submissions, Reviewers can access only assigned submissions, and Admins can manage the entire system. This ensures secure, stateless, and role-specific access throughout the platform.

7.6 Role-Based User Flow

7.6.1 Author (User) Flow

- Logs into the **Client Interface** using email and password
- Uploads research paper (PDF) through the submission form
- Views and tracks submission status (Pending, Under Review, Accepted, Rejected, Revision Required)
- Receives reviewer feedback and decision updates
- Can access and view **only their own submissions**

7.6.2 Reviewer Flow

- Logs into the **Admin Interface** (Reviewer Dashboard)
- Views the list of papers assigned by the Admin
- Downloads the submission for evaluation
- Provides feedback and submits a recommendation (Accept, Reject, or Revision Required)
- Can interact **only with assigned submissions**

7.6.3 Admin Flow

- Logs into the **Admin Interface** (Admin Dashboard)
- Views all submissions in the system
- Assigns reviewers to submissions
- Updates the status of submissions based on reviewer feedback
- Manages users, reviewers, and registrations
- Has complete control over the entire workflow and system operations

8. Work Done

The development work for the project *"IPDIMS - Conference website management system"* has progressed to the completion of a fully functional and deployed web application. The system was developed independently, without referencing or modifying the previous existing version, although the older system helped in understanding the problem context. The project was completed through iterative development stages including requirement refinement, design of architecture, backend implementation, database modeling, frontend development, and real-time testing with live deployment.

8.1 Requirement Understanding and Workflow Structuring

The project began with understanding the manual challenges involved in managing conference paper submissions, including scattered communication, difficulty in assigning reviewers, and lack of transparent tracking for authors. After identifying these issues, a structured workflow was defined consisting of three distinct roles - Author, Reviewer, and Admin. Each role was mapped to clear responsibilities: authors submit and track papers, reviewers evaluate submissions and provide feedback, and admins supervise the entire process. This requirement

phase ensured that the system addressed real conference needs rather than just implementing technical features.

8.2 System Architecture and Backend Implementation

A three-tier architecture was designed and implemented using React (for frontend and admin dashboards), Node.js with Express.js (for backend API services), and MongoDB (for persistent storage). Secure JWT-based authentication with role-based access control was implemented to ensure that authors, reviewers, and admins access only their relevant interfaces and data. Multer was used to process uploads, and Cloudinary was integrated to store documents securely on the cloud. The backend consists of modular controllers and routes handling user authentication, paper submissions, reviewer assignment, feedback processing, and status updates.

8.3 Frontend and Admin Dashboard Development

Two independent web interfaces were developed using React and Vite - one functioning as the Author Portal and the other as the Admin/Reviewer Dashboard. The Author Portal allows users to register, submit papers, track submission status, view decisions, and see reviewer feedback. The Admin Dashboard provides functionality to create reviewer accounts, view all submissions, assign papers to reviewers, change submission status, and manage user registrations. The dashboard UI was designed to be minimal and intuitive, providing clarity and smooth navigation during critical tasks such as reviewer assignment and decision updates.

8.4 Email Integration, Status Automation, and Testing

Nodemailer was used to implement automated email notifications that alert users at key moments, such as successful submission, reviewer assignment, and acceptance or revision decisions. Real-time paper status updates were successfully implemented so that authors can monitor progress without requiring manual communication. Functional and integration testing was carried out to ensure proper backend communication with the frontend applications, safe file uploads, and secure handling of user data. The integrated system was tested end-to-end to validate flow consistency across all three user roles.

8.5 Deployment and Final Output

The backend was deployed on Railway and the frontend dashboards were deployed on Vercel, resulting in a stable, cloud-hosted system accessible from any device. The final output is a fully operational conference management system where authors can upload research papers, reviewers can evaluate assigned submissions, and admins can manage the entire workflow seamlessly. The system eliminates manual coordination and replaces it with a structured, automated, and traceable process.

9. Work to Be Done

In the next phase of the project, the system will be improved to make it more refined and conference-ready. The file upload mechanism will be optimized to handle larger files more efficiently and provide better feedback during uploads. A payment gateway will be integrated so that authors of accepted papers can directly complete registration through the system. The user interface will be redesigned to be visually appealing, modern, and fully responsive across all devices including tablets and mobile phones. Additional enhancements will be made to simplify navigation, improve loading speed, and make the workflow more intuitive for first-time users. Further improvements may also be introduced based on user feedback once the system is used in a real conference environment.

10. Conclusion

The project titled **"IPDIMS – Conference Website Management System"** successfully achieves the objective of digitizing and streamlining the academic paper submission and review workflow. The system addresses the common challenges faced in manual processes such as scattered submissions across emails, delayed communication, and lack of status transparency by providing an integrated platform where Authors, Reviewers, and Admins operate within clearly defined roles.

Using a three-tier architecture with React (frontend), Node.js + Express (backend), and MongoDB (database), the system enables secure authentication, structured submission handling, reviewer assignment, and centralized decision tracking. Features such as real-time status updates, reviewer feedback, role-based access control (JWT), automated email notifications, and cloud storage for uploaded files make the workflow efficient and traceable. The separation of the Author portal and the combined Admin–Reviewer dashboard ensures that users interact only with features relevant to their responsibilities, improving usability and reducing complexity.

Overall, the implementation demonstrates how modern web technologies can replace manual, error-prone processes with a transparent and automated digital workflow. IPDIMS provides a scalable framework that can be extended for academic conferences, journals, and institutional documentation systems in the future. By improving turnaround time, ensuring accountability, and simplifying coordination, the system contributes to a more structured and efficient review ecosystem.

References:

1 Belchior, M., Schwabe, D., & Parreiras, F. S. (2012). **Role-based access control for model-driven web applications.** In M. Brambilla, T. Tokuda, & R. Tolksdorf (Eds.), *ICWE 2012: International Conference on Web Engineering* (Lecture Notes in Computer Science, Vol. 7387, pp. 106–120). Springer. https://doi.org/10.1007/978-3-642-31753-8_8

2 Auth0 Documentation. (2024). *Introduction to JSON Web Tokens (JWT): Secure Authentication for Modern Applications*. Retrieved from https://auth0.com/docs/secure/tokens/json-web-tokens

3 Cloudinary Documentation. (2024). File Upload and Cloud Storage for Web Applications. Retrieved from https://cloudinary.com/documentation

4 MongoDB Inc. (2024). *MongoDB NoSQL Database Architecture and Document Storage*. Retrieved from https://www.mongodb.com/docs